Model Evaluation and Validation

Md. Aminul Islam Shazid

Outline

1. Why we evaluate models
2. Overfitting and underfitting
3. The bias-variance trade-off
4. Validation set and cross-validation
5. Classification metrics
6. Model selection and comparison

Why we evaluate models

Why evaluate a model?

- The goal of supervised learning is to predict well on *new* observations $(\mathbf{x}^*, y^*)$, not on the data used for fitting
- Error on the fitting (training) data is an over-optimistic estimate of error on new data
- We need honest estimates of performance on data the model has not seen
- Two related tasks:
  - **Model assessment**: how well does the final model perform?
  - **Model selection**: which model, or which level of flexibility, should we choose?

Training error vs. test error

- Regression, with n training observations:

$$\mathrm{MSE}_{\text{train}} = \frac{1}{n}\sum_{i=1}^{n}\left(y^{(i)} - \hat{f}(\mathbf{x}^{(i)})\right)^2, \qquad \mathrm{MSE}_{\text{test}} = \mathbb{E}\left[(y^* - \hat{f}(\mathbf{x}^*))^2\right]$$

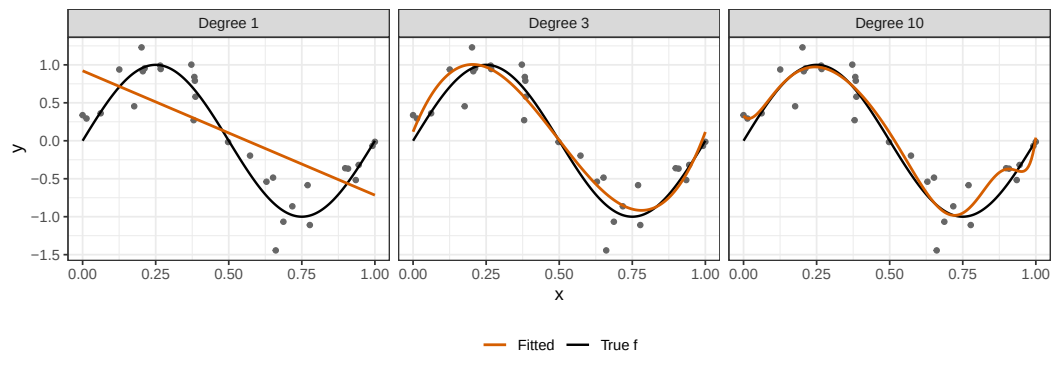
- $\mathrm{MSE}_{\text{test}}$ is estimated on a held-out set $\mathcal{T}$ of m observations: $\frac{1}{m}\sum_{i\in\mathcal{T}}\left(y^{(i)} - \hat{f}(\mathbf{x}^{(i)})\right)^2$
- Classification: error rate $\frac{1}{n}\sum_{i=1}^{n} I(y^{(i)} \neq \hat{y}^{(i)})$, with the test version defined analogously
- Fitting methods minimize training error, so a small training error does *not* guarantee a small test error

Overfitting and underfitting

Underfitting and overfitting

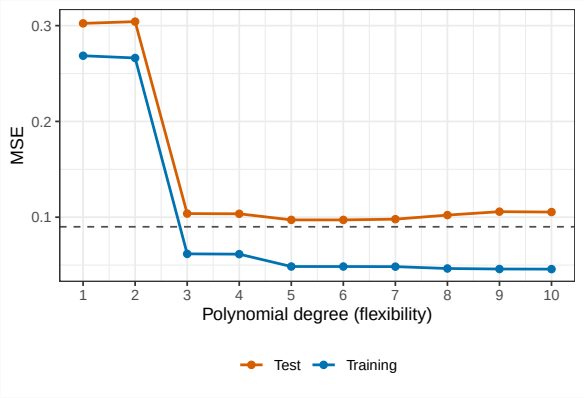
- **Underfitting**: the model is too simple to capture the structure in f
  - High error on the training data *and* on the test data
- **Overfitting**: the model is so flexible that it fits the noise ϵ in the training sample
  - Very low training error, but much higher test error
- Diagnosis: compare training and test (or validation) error
  - Both high $\Rightarrow$ underfitting; large gap between them $\Rightarrow$ overfitting

Example: polynomial fits to simulated data



- Data: $y = \sin(2\pi x) + \epsilon$ with $\epsilon \sim N(0, 0.3^2)$ and $n = 30$; fits are polynomials of degree 1, 3 and 10
- Degree 1 underfits; degree 3 is close to the truth; degree 10 chases the noise

Training error vs. test error as flexibility grows



- Training MSE decreases as the degree increases
- Test MSE is U-shaped: it first falls, then rises
- The dashed line is σ^2, the error no model can beat
- Choosing the model with the smallest training MSE would select the most overfit model

What to do about under- and overfitting

If underfitting

- Use a more flexible model
- Add features or interactions
- Reduce regularization

If overfitting

- Use a simpler model
- Collect more training data
- Remove features or add regularization

- Both remedies move the model along the flexibility axis; finding the right point requires an honest estimate of test error

The bias-variance trade-off

Bias-variance decomposition: setup

- Assume $y^* = f(\mathbf{x}^*) + \epsilon$ with $\mathbb{E}[\epsilon] = 0$, $\mathrm{Var}(\epsilon) = \sigma^2$, and ϵ independent of the training data
- The estimate $\hat{f}$ is computed from a random training set $\mathcal{D}$, so $\hat{f}(\mathbf{x}^*)$ is itself a random variable
- Expected test MSE at a fixed point $\mathbf{x}^*$, taken over training sets $\mathcal{D}$ and noise ϵ:

$$\mathbb{E}\big[(y^* - \hat{f}(\mathbf{x}^*))^2\big]$$

- Goal: split this quantity into interpretable pieces

Bias-variance decomposition: step 1 (noise)

Substitute $y^* = f(\mathbf{x}^*) + \epsilon$ and expand the square (writing f and $\hat{f}$ for $f(\mathbf{x}^*)$ and $\hat{f}(\mathbf{x}^*)$):

$$\begin{aligned}\mathbb{E}[(y^* - \hat{f})^2] &= \mathbb{E}[(f - \hat{f} + \epsilon)^2] \\ &= \mathbb{E}[(f - \hat{f})^2] + 2\,\mathbb{E}[\epsilon\,(f - \hat{f})] + \mathbb{E}[\epsilon^2]\end{aligned}$$

- ϵ is independent of $\hat{f}$ and has mean zero, so the cross term is $2\,\mathbb{E}[\epsilon]\,\mathbb{E}[f - \hat{f}] = 0$
- $\mathbb{E}[\epsilon^2] = \mathrm{Var}(\epsilon) = \sigma^2$

$$\mathbb{E}[(y^* - \hat{f})^2] = \mathbb{E}[(f - \hat{f})^2] + \sigma^2$$

Bias-variance decomposition: step 2 (bias and variance)

Let $\bar{f} = \mathbb{E}[\hat{f}(\mathbf{x}^*)]$. Add and subtract $\bar{f}$ inside the remaining term:

$$\begin{aligned}\mathbb{E}[(f - \hat{f})^2] &= \mathbb{E}[((f - \bar{f}) + (\bar{f} - \hat{f}))^2] \\ &= (f - \bar{f})^2 + 2\,(f - \bar{f})\,\mathbb{E}[\bar{f} - \hat{f}] + \mathbb{E}[(\bar{f} - \hat{f})^2]\end{aligned}$$

- $f - \bar{f}$ is a constant and $\mathbb{E}[\bar{f} - \hat{f}] = 0$, so the cross term vanishes

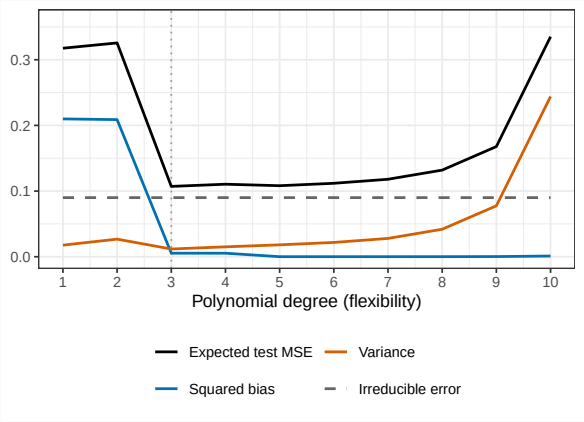
$$\mathbb{E}[(y^* - \hat{f}(\mathbf{x}^*))^2] = \underbrace{\mathrm{Var}(\hat{f}(\mathbf{x}^*))}_{\text{variance}} + \underbrace{[\mathrm{Bias}(\hat{f}(\mathbf{x}^*))]^2}_{\text{squared bias}} + \underbrace{\sigma^2}_{\text{irreducible}}$$

with $\mathrm{Bias}(\hat{f}(\mathbf{x}^*)) = \mathbb{E}[\hat{f}(\mathbf{x}^*)] - f(\mathbf{x}^*)$.

Interpreting the three terms

- **Variance**: how much $\hat{f}(\mathbf{x}^*)$ changes if we refit on a different training set
  - Flexible methods follow the training sample closely, so their variance is high
- **Bias**: error from approximating the true f by a simpler model class
  - A straight line fitted to a curved f is wrong on average, however much data we have
- **Irreducible error** σ^2: noise in y^*; a lower bound on test MSE
- More flexibility typically lowers bias and raises variance; the test MSE depends on which changes faster

The trade-off in simulation



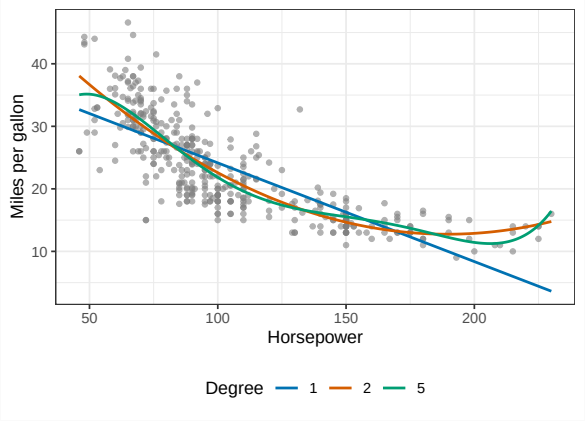
- Estimated from 500 simulated training sets, averaged over $x^* \in [0, 1]$
- Squared bias falls and variance rises with the polynomial degree
- Expected test MSE is their sum plus σ^2
- The best degree depends on f, n and σ^2; more data moves the optimum towards more flexible models

Bias-variance for classification

- With the 0–1 loss the decomposition is not additive in the simple form above, but the same qualitative behaviour holds.
- Flexible classifiers have low bias and high variance, while rigid ones have the reverse.
- The test error rate is again U-shaped in flexibility, so it must be estimated without using the training error.

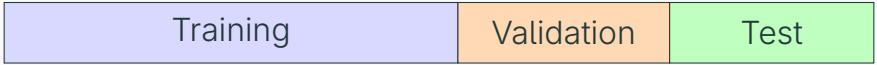
Validation set and cross-validation

Running example: mpg vs. horsepower (`Auto`)



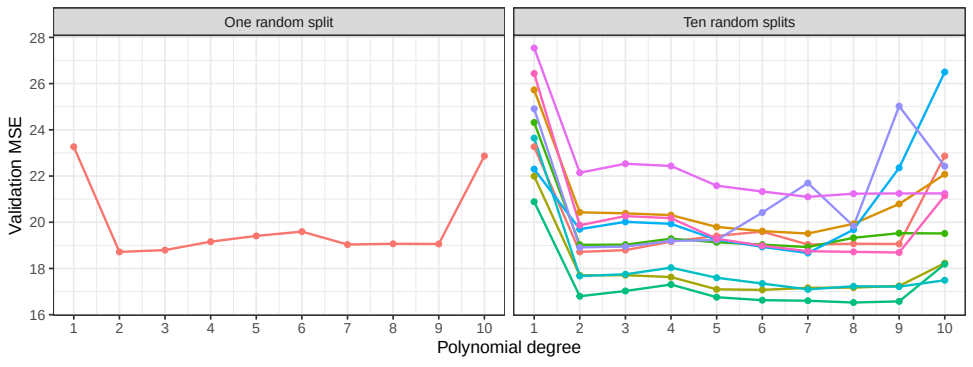
- $n = 392$ cars; response mpg, predictor horsepower
- Model: polynomial regression $\hat{f}(x) = \hat{\beta}_0 + \hat{\beta}_1 x + \cdots + \hat{\beta}_d x^d$
- The degree d controls flexibility; which d should we use?

Training, validation and test sets



- **Training set**: used to fit the parameters of each candidate model
- **Validation set**: used to compare candidates and tune flexibility
- **Test set**: used once, at the end, to assess the chosen model; never used for any choice
- If test data influence any decision, the test error is no longer an honest estimate

Validation set approach



- Randomly split the data into two halves; fit on one, compute the MSE on the other
- The estimate, and even the best degree, changes from split to split (right panel)
- Only part of the data is used for fitting, so the validation MSE tends

Leave-one-out cross-validation (LOOCV)

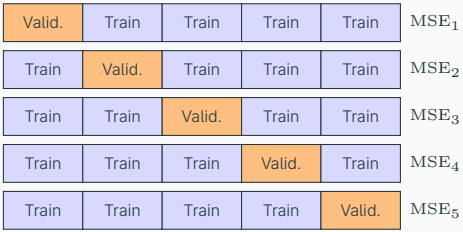
- Hold out observation i, fit on the remaining $n-1$, predict $y^{(i)}$; repeat for $i = 1, \dots, n$

$$\mathrm{CV}_{(n)} = \frac{1}{n}\sum_{i=1}^{n} \mathrm{MSE}_i, \qquad \mathrm{MSE}_i = \left(y^{(i)} - \hat{y}^{(i)}_{(-i)}\right)^2$$

- $\hat{y}^{(i)}_{(-i)}$ is the prediction for $\mathbf{x}^{(i)}$ from the model fitted without observation i
- Each fit uses $n-1$ points, so the bias of the estimate is small, and there is no randomness in the splits
- For least squares with hat values h_i, one fit suffices:

$$\mathrm{CV}_{(n)} = \frac{1}{n}\sum_{i=1}^{n} \left(\frac{y^{(i)} - \hat{y}^{(i)}}{1 - h_i}\right)^2$$

k-fold cross-validation



- Randomly partition the data into k folds $F_1, \ldots, F_k$ of about equal size ($k = 5$ shown)
- For each j: fit on all folds except F_j, evaluate on F_j

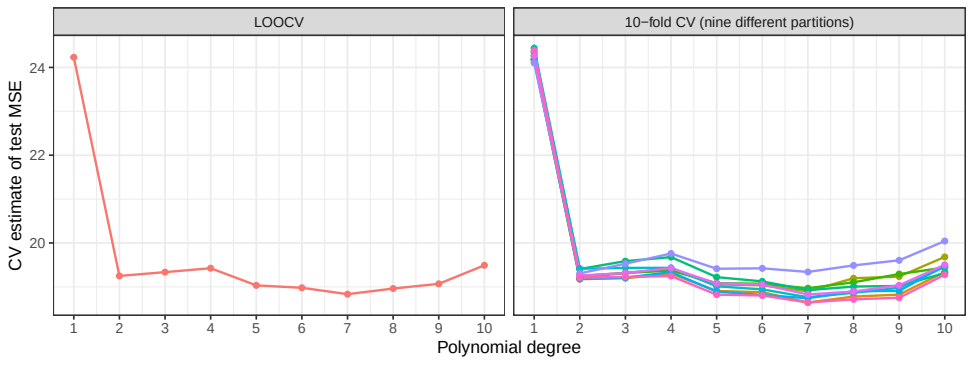
$$\text{MSE}_j = \frac{1}{|F_j|} \sum_{i \in F_j} \left(y^{(i)} - \hat{f}^{(-j)}(\mathbf{x}^{(i)})\right)^2, \qquad \text{CV}_{(k)} = \frac{1}{k} \sum_{j=1}^{k} \text{MSE}_j$$

Validation set, LOOCV and k-fold compared

Method	Model fits	Bias of estimate	Variance of estimate
Validation set	1	high	high
LOOCV ($k = n$)	n	lowest	high
k-fold ($k = 5, 10$)	k	low to moderate	moderate

- LOOCV training sets are nearly identical, so its n fitted models are highly correlated and their average has high variance
- Smaller k means more bias (less training data per fit) but less variance and less computation
- Empirically, $k = 5$ or $k = 10$ gives estimates that suffer neither from excessive bias nor from high variance

Cross-validation on `Auto`



- CV error drops from about 24.2 (linear) to about 19.2 (quadratic), with no clear gain beyond degree 2
- Ten-fold CV curves from different random partitions agree closely, unlike the validation set curves
- The location of the minimum matters more than the exact level of the

Cross-validation for classification

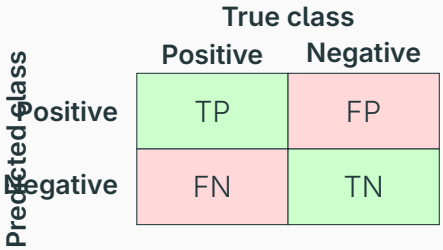
- Replace the squared error by the misclassification indicator

$$\mathrm{Err}_j = \frac{1}{|F_j|} \sum_{i \in F_j} I(y^{(i)} \neq \hat{y}^{(i)}), \qquad \mathrm{CV}_{(k)} = \frac{1}{k} \sum_{j=1}^{k} \mathrm{Err}_j$$

- Any other metric (precision, recall, AUC, ...) can be computed per fold and averaged in the same way

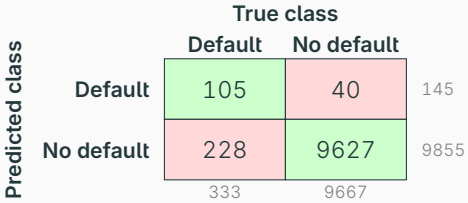
Classification metrics

The confusion matrix



- Rows: predicted class; columns: true class (the layout used in ISLP; some software transposes it)
- TP, TN: correct predictions; FP (false alarm), FN (miss): errors
- The positive class is the one we care about detecting
- For $K > 2$ classes the matrix is $K \times K$

Example: `Default` data, threshold 0.5



- Logistic regression of `default` on `student`, `balance`, `income`; $n = 10{,}000$ with 333 defaulters
- Predict "default" when $\hat{p}(\mathbf{x}^*) > 0.5$
- Matrix computed on the training data, as in ISLP; on new data expect somewhat worse numbers

Accuracy and error rate

$$\text{Accuracy} = \frac{TP + TN}{TP + FP + FN + TN}, \qquad \text{Error rate} = 1 - \text{Accuracy}$$

- `Default`: accuracy $= (105 + 9627)/10000 = 97.3\%$
- A classifier that always predicts “no default” has accuracy $9667/10000 = 96.7\%$ while catching no defaulter at all
- With imbalanced classes, accuracy mostly measures how large the majority class is
- Use accuracy when classes are roughly balanced and both error types cost about the same

Precision, recall and specificity

$$\text{Precision} = \frac{TP}{TP + FP} \quad \text{flagged cases that are truly positive}$$

$$\text{Recall (sensitivity)} = \frac{TP}{TP + FN} \quad \text{true positives that we find}$$

$$\text{Specificity} = \frac{TN}{TN + FP} \quad \text{true negatives that we clear}$$

- `Default` at threshold 0.5: precision $= 105/145 \approx 0.72$, recall $= 105/333 \approx 0.32$, specificity $= 9627/9667 \approx 0.996$
- High accuracy and high specificity hide that two thirds of the defaulters are missed
- False positive rate $= FP/(FP + TN) = 1 -$ specificity

Precision vs. recall: intuition

Precision: “When I raise an alarm, how often am I right?”

- Spam filter: a real email sent to the spam folder is costly, a few spam emails in the inbox are not
- Fraud alerts reviewed by a small team: every false alert wastes investigator time

Recall: “Of all real cases, how many did I catch?”

- Cancer screening: a missed case is far worse than an extra follow-up test
- Flood or earthquake early warning: a missed event is catastrophic, a false alarm is an inconvenience

- Targeting cash transfers to poor households: low recall excludes eligible households, low precision leaks money to the non-poor
- Specificity matters when clearing negatives is the goal, e.g. avoiding unnecessary treatment

Which metric should I use?

Situation	Metric to emphasize
Balanced classes, similar error costs	Accuracy
False positives are costly	Precision, specificity
False negatives are costly	Recall (sensitivity)
Rare positives, both errors matter	F_1 score, precision–recall curve
Comparing classifiers over all thresholds	ROC curve and AUC

- Decide which error is more expensive *before* looking at the numbers
- Always compare against a trivial baseline such as the majority-class classifier

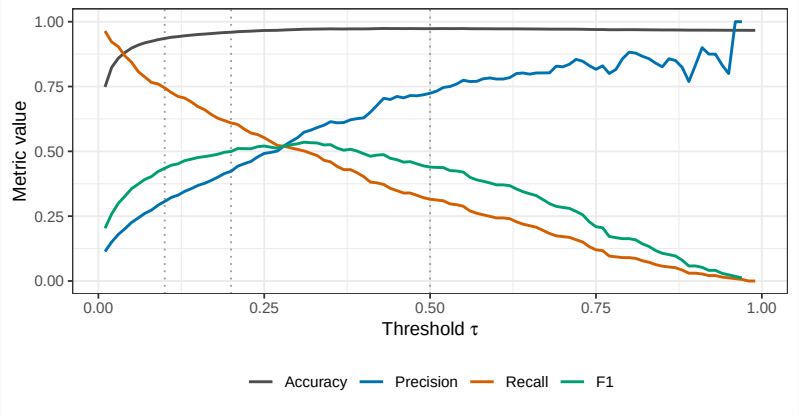
Changing the threshold

- Classify as positive when $\hat{p}(\mathbf{x}^*) > \tau$; lowering τ flags more cases

τ	TP	FP	FN	TN	Accuracy	Precision	Recall	F_1
0.5	105	40	228	9627	0.973	0.724	0.315	0.439
0.2	203	277	130	9390	0.959	0.423	0.610	0.499
0.1	248	560	85	9107	0.936	0.307	0.745	0.435

- Recall rises as τ falls, while precision and accuracy fall: the two error types trade off
- The right τ depends on the relative costs of missing a defaulter and of rejecting a good customer

Metrics as a function of the threshold



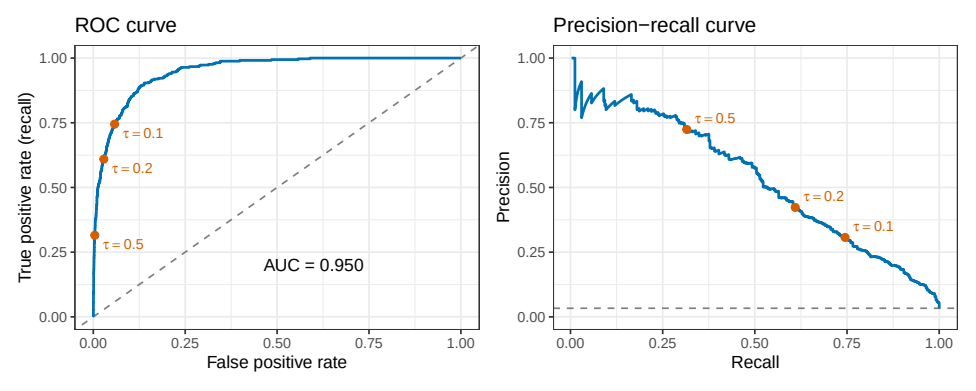
- Recall decreases monotonically in τ; precision tends to increase, but not monotonically
- Accuracy is highest at large τ simply because most cases are

The F_1 score

$$F_1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} = \frac{2\,TP}{2\,TP + FP + FN}$$

- Harmonic mean of precision and recall: it is low unless *both* are reasonably high
- Ignores TN, so it is not inflated by a large negative class
- $F_\beta = (1 + \beta^2)\,\dfrac{\text{Precision} \cdot \text{Recall}}{\beta^2\,\text{Precision} + \text{Recall}}$ weights recall β times as much as precision
- For $K > 2$ classes, compute the metric per class and average (macro-average), or pool the counts (micro-average)

ROC curve, precision–recall curve and AUC



- ROC: true positive rate (recall) against false positive rate as τ varies; AUC ≈ 0.95 here (training data)
- AUC is the probability that a random positive receives a higher $\hat{p}$ than a random negative; 0.5 is chance, 1 is perfect

Model selection and comparison

A workflow for selection and assessment

1. Split off a test set and set it aside
2. On the training data, use CV to compare candidate models and to tune hyperparameters such as the polynomial degree
3. Refit the chosen model on the entire training set
4. Evaluate it once on the test set and report that number

- The CV error of the winning model is optimistic, because it was selected for having the lowest CV error

Adjusted R^2, AIC and BIC

- Penalize the training fit for model size instead of using held-out data; d = number of predictors, p = number of estimated parameters

$$R^2_{\text{adj}} = 1 - \frac{\text{RSS}/(n-d-1)}{\text{TSS}/(n-1)}, \qquad \text{AIC} = -2\log\hat{L} + 2p, \qquad \text{BIC} = -2\log\hat{L} + p\log n$$

Degree	R^2_{adj}	AIC	BIC
1	0.605	2363.3	2375.2
2	0.686	2274.4	**2290.2**
3	0.686	2275.5	2295.4
4	0.686	2276.1	2299.9
5	0.693	2268.7	2296.5
6	**0.695**	**2266.7**	2298.4

Adjusted R^2, AIC, BIC or cross-validation?

- Lower AIC and BIC are better; higher R^2_{adj} is better
- BIC has the heavier penalty ($\log n > 2$ for $n \geq 8$) and picks the quadratic; AIC and R^2_{adj} favour a higher degree
- These criteria assume a likelihood-based model and a well-defined parameter count, and they rely on the training fit
- CV assumes little about the model and works for any method and any metric, at the cost of refitting the model several times

Comparing models fairly

- Compare all models on exactly the same folds, so that differences are not due to different splits
- Use the metric that matches the goal (MSE, recall, F_1, AUC, ...), and include a trivial baseline
- Look at per-fold differences between two models and their standard error, not just the two means
- A small difference relative to the SE is not evidence that one model is better
- Report the final performance on the untouched test set

Summary

- Training error is optimistic; estimate test error with held-out data
- Test error is U-shaped in flexibility:
  $\mathbb{E}[(y^* - \hat{f}(\mathbf{x}^*))^2] = \mathrm{Var} + \mathrm{Bias}^2 + \sigma^2$
- k-fold CV ($k = 5$ or 10) is the default tool for model assessment and selection
- Classification: read the confusion matrix, then choose accuracy, precision, recall, F_1 or AUC according to the costs of the two error types
- Select with CV (or AIC/BIC) on the training data, and touch the test set only once

Questions?